

Applied NLP

DATA 641

Lecture 1 — Python tutorial for NLP

Python Bootcamp Phase

Basics	String Manipulation	Functions
Variables	String constants	Making functions
Conditionals	Custom String Formatting	Proper use of functions
Loops and Iterators	Format String Syntax	Helper functions
Reading/Writing Files	Template strings	
Lists, Sets, Dictionaries		
Numpy		

Data Types

There are only a few native data types in Python:

1. Booleans are either `True` or `False`

- e.g. `my_bool = False`

2. Numbers: integers (`0` and `42`) or floats (`0.1` and `12.245`)

- e.g. `my_var = 2.567`

3. Strings are sequences of Unicode characters, e.g. an html document.

- e.g. `text = "Hello there"`

Data Types

4. (Bytes and byte arrays, e.g. a jpeg image file.)

5. Lists are ordered sequences of values.

- e.g.

```
>>myList = [1, 2, 3]
```

```
>>myList[0] = 2
```

```
myList = [2, 2, 3]
```

6. Tuples are ordered, immutable sequences of values.

- e.g.

```
>>myTup = (1, 2, 3)
```

```
>>myTup[0] = 2
```

```
Error
```

Data Types

7. Sets are unordered bags of values.

- e.g.

```
>>mySet = {2, 1, 3}
```

```
mySet = {1, 2, 3}
```

8. Dictionaries are unordered bags of key-value pairs.

- e.g.

```
>>myDictionary = {'a': 2, 'b' : 3}
```

```
myDictionary['b']
```

```
3
```


Conditionals

Follows order: `if -> elif (else if) -> else`

```
In [1]: # Example

myNum = 4

if myNum < 0: #if <condition> is True
    print('Negative')
elif myNum == 0:
    print('Zero')
else:
    print('Positive')
```

Positive

While Loops

Executes commands as long as a `<condition>` is `True`

In [2]:

```
a = 0
while a < 5:
    print(a)
    a = a + 1 #shorthand: a+=1
```

#Question: What happens when we remove `a = a + 1`?

```
0
1
2
3
4
```

Creating Functions

In [3]: `def addTwo(x): # input int (x), output int (y)`
 `y = x + 2`
 `return y`

`def generateTuple(x):`
 `y = x*2`
 `z = y*2`
 `return (y,z)`

`print(addTwo(2))`
`print(generateTuple(2))`

4

(4, 8)

Lists and Indexing

Recall that lists are ordered sequences of values.

Lists can be accessed via indexing:

```
In [4]: myList = ['a', 'b', 'c', 'c']

print(myList[0]) # note zero indexing

print(myList[-1]) # negative indexing!

print(myList[1:3]) # ranges: to last digit (not inclusive)
```

a

c

['b', 'c']

Commands to use with Lists

```
myList = ['a', 'b', 'c', 'd']
```

Length of list via `len`

```
>> len(myList)
```

Check for inclusion via the `in` keyword

```
>> 'b' in myList
```

Grow lists via `append`

```
>> myList.append('e')
```

Arithmetic Operations With Lists

```
myList = ['a', 'b']
```

You can use the + and * operators to grow lists. However you cannot use the -and / operators to shrink lists.

```
In [5]: myList = ['a', 'b']  
  
        print(myList + ['c'])  
  
        print(myList * 2)  
  
['a', 'b', 'c']  
['a', 'b', 'a', 'b']
```


List Methods

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Sets

<code>len(s)</code>	Return the number of elements in sets(cardinality of s)
<code>x in s / x not in s</code>	Test x for membership or non-membership in s
<code>isdisjoint(other)</code>	Return True if the set has no elements in common with other
<code>issubset(other)</code>	Test whether every element in the set is in other
<code>issuperset(other)</code>	Test whether every element in other is in the set

Set Operations

<code>union(other)</code>	Return a new set with elements from the set and all others
<code>intersection(other)</code>	Return a new set with elements common to the set and all others
<code>difference(other)</code>	Return a new set with elements in the set that are not in the others
<code>symmetric_difference(other)</code>	Return a new set with elements in either the set or other but not both

Casting

"Casting" is the act of converting a variable from one type to another

Examples

1. You can cast from int to float:

- `float(1) = 1`

2. You can cast a list to a set:

- `set([1,1,2,3]) = {1,2,3} # where's the other 1?`

Dictionaries

Recall that dictionaries are unordered bags of key-value pairs.

```
myDict = {'a':1, 1:2, ('a',2):1}
```

Keys

- unique within a dictionary while values may not be
- must be of an immutable data type such as strings, numbers, or tuples

Values

- Can be of any type

Dictionary Methods

Method	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and values
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs

Strings Constants

This is an NLP class, and you will be dealing a **lot** with strings. Let's go over some basic constants from the string module:

<code>string.ascii_lowercase</code>	'abcdefghijklmnopqrstuvwxyz'
<code>string.ascii_uppercase</code>	'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
<code>string.digits</code>	The string '0123456789'
<code>string.punctuation</code>	!"#\$ % & @ etc
<code>string.whitespace</code>	space, tab, linefeed, return, form feed, and vertical tab.

Useful String Methods

Method	Description
<code>capitalize()/lower()</code>	Capitalize (lower) all letters in string
<code>isalpha()</code>	Return True if all characters in the string are alphabetic and there is at least one character
<code>isnumeric()</code>	Return True if all characters in the string are numeric characters, and there is at least one character, False otherwise
<code>split()</code>	Return a list of the words in the string, using sep as the delimiter string
<code>join()</code>	Return a string which is the concatenation of the strings in iterable
<code>strip()</code>	Return a copy of the string with the leading and trailing characters removed
<code>replace()</code>	Return a copy of the string with all occurrences of substring old replaced by new

String Formatting

The `format()` method enables variable substitutions and value formatting

Some examples:

```
In [6]: print('{0}, {1}, {2}'.format('a', 'b', 'c'))  
  
        print('{} , {} , {}'.format('a', 'b', 'c'))  
  
        print('{0}{1}{0}'.format('abra', 'cad'))  
  
        print('Coordinates: {lat}, {lon}'.format(lat='37.24N', lon='-115.81W'))
```

a, b, c

a, b, c

abracadabra

Coordinates: 37.24N, -115.81W